

Rapport de stage de fin d'études

Développement de fonctionnalités full-stack sur une plateforme SaaS de santé au travail

Padoa — Paris

Auteur :
BOUNOUA Ilyas

École Centrale de Nantes — Systèmes d'Information

Stage ingénieur de six mois, avril – septembre 2026

Version portfolio — publique

Ce document est une édition expurgée d'un rapport de stage rédigé pour Padoa, éditeur de logiciels de santé au travail. Les noms internes de dépôts, de modules, de bases de données et d'interrupteurs de configuration, les identifiants de tickets, les références à la documentation interne, les chiffres commerciaux, les noms de clients, les captures d'écran d'outils internes ainsi que les noms des collègues et des encadrants ont tous été retirés. Il en reste le raisonnement d'ingénierie : les problèmes, les arbitrages de conception, les compromis et leurs conséquences. Les objets techniques sont donc décrits par leur fonction plutôt que nommés.

Table des matières

I	Introduction & intégration	3
I.1	Intégration technique et environnement de développement	3
I.2	L'écosystème frontend et les environnements	3
I.3	L'IA au service de la montée en compétences et de l'analyse	4
I.3.1	Exploration du code et apprentissage	4
I.3.2	Soutien à la phase d'analyse	4
I.3.3	Professionnalisme et bonnes pratiques	4
II	Macro-contexte & analyse de marché	5
II.1	Le cadre français de la santé au travail (SPSTI / SPSTA)	5
II.2	Positionnement sur le marché et stratégie d'expansion	5
II.3	Organisation interne et place de mon équipe	6
III	Méthodologie agile & opérations d'ingénierie	7
III.1	Rituels agiles et cycles de sprint	7
III.2	Rotations opérationnelles	7
III.2.1	Le rôle de support d'astreinte	7
III.2.2	Le Scrum Master tournant par itération	8
III.3	Cadre de découpage technique	8
IV	Objectifs du stage & feuille de route	9
IV.1	La mission : cadrage du sujet	9
IV.2	Jalons initiaux et calendrier	9
IV.3	Attentes fonctionnelles générales	9
V	Étude de cas : identification des doublons entre entités via un identifiant d'entreprise commun	10
V.1	Phase 1 : le système existant et ses limites	10
V.2	Phase 2 : choix de conception	10
V.3	Phase 3 : ce que j'ai implémenté	10
V.4	Phase 4 : validation et regard critique	11
VI	Étude de cas : résolution et publication de l'assistant social d'un secteur sur deux portails	13
VI.1	Phase 1 : le système existant et ses limites	13
VI.2	Phase 2 : choix de conception	13
VI.3	Phase 3 : ce que j'ai implémenté	14
VI.4	Phase 4 : validation et regard critique	14
VII	Étude de cas : vérification et enrichissement des prospects créés automatiquement	16
VII.1	Phase 1 : le système existant et ses limites	16
VII.2	Phase 2 : choix de conception	16
VII.3	Phase 3 : ce que j'ai implémenté	17
VII.4	Phase 4 : validation et regard critique	17
VIII	Étude de cas : révocation en masse des accès d'un utilisateur sur tous ses adhérents	19
VIII.1	Phase 1 : le système existant et ses limites	19
VIII.2	Phase 2 : choix de conception	19
VIII.3	Phase 3 : ce que j'ai implémenté	20
VIII.4	Phase 4 : validation et regard critique	20
IX	Conclusion et bilan personnel	23
IX.1	Bilan professionnel & résultats collectifs	23
IX.2	Développement personnel & posture d'ingénieur	23

I Introduction & intégration

Ce rapport présente le stage de fin d'études que j'ai effectué d'avril à septembre 2026 chez Padoa, entreprise française basée à Paris qui édite une plateforme SaaS destinée aux services de prévention et de santé au travail. Dernière étape du diplôme d'ingénieur de l'École Centrale de Nantes, spécialité Systèmes d'Information, cette mission de six mois marque le passage de la formation académique à la pratique professionnelle de l'ingénierie. Le stage a porté sur le développement de fonctionnalités full-stack au sein du pôle produit responsable du cycle de vie des adhérents et des prospects.

Les objectifs principaux de cette mission étaient de concevoir et d'implémenter des composants full-stack prêts pour la production, d'optimiser les pipelines qui maintiennent l'index de recherche backend synchronisé, et de résorber une dette technique historique qui limitait la montée en charge de la plateforme.

Note de vocabulaire : Deux termes reviennent tout au long de ce rapport, et chacun n'y est employé que dans un seul sens. Un **adhérent** est une entreprise qui a rejoint un service de santé au travail et dont ce service assure le suivi des salariées. Un **prospect** est une entreprise qui ne l'a pas encore rejoint. Le service peut identifier et créer lui-même la fiche, que ce soit par une démarche commerciale ou automatiquement à partir d'une déclaration administrative, ou bien l'entreprise peut la créer elle-même en remplissant un questionnaire d'éligibilité. Le mot *lead* n'est volontairement pas employé comme synonyme, car chez Padoa il relève du vocabulaire commercial allemand et non du vocabulaire français.

I.1 Intégration technique et environnement de développement

La première phase du stage a été consacrée à une intégration technique rigoureuse, destinée à aligner mon environnement de développement sur les standards d'ingénierie de Padoa. Ce processus s'est appuyé sur un dépôt d'amorçage qui sert de point d'entrée unique à tous les développeurs, à partir duquel l'outillage est installé et les dépôts secondaires sont clonés.

Les étapes clés de cette mise en place ont été :

- **IDE et outils de développement :** l'adoption d'un IDE assisté par IA comme éditeur principal, avec une licence gérée par SSO, et l'installation des outils d'inspection sur lesquels l'équipe s'appuie en dehors de l'éditeur : l'extension navigateur Redux DevTools pour suivre l'état client, et un client Elasticsearch dans le navigateur pour lire directement les index de recherche.
- **Connectivité :** la configuration du VPN d'entreprise et de l'extension navigateur interne utilisée pour naviguer entre les environnements clients et les outils internes.
- **Vue d'ensemble de l'architecture :** la participation aux sessions de l'académie d'ingénierie interne, qui proposaient des plongées approfondies dans les architectures frontend et backend. Ces présentations m'ont permis de comprendre comment sont structurés les différents workspaces Yarn et comment la bibliothèque centrale partagée soutient les applications principales.

En pratique, cette configuration permettait de valider chaque fonctionnalité sur trois couches depuis un seul poste : l'IDE pour le code source et les diffs, l'application elle-même servie en local (le front-end branché sur un serveur backend local et sa base de données, tous deux démarrés par les scripts d'amorçage), et les panneaux Redux DevTools et Elasticsearch du navigateur pour inspecter l'état client et l'index de recherche. Cette boucle à trois couches est réutilisée plus loin pour vérifier les comptes de doublons de la section V.

I.2 L'écosystème frontend et les environnements

Une part importante de mon immersion technique a consisté à comprendre les patterns frontend propres à la plateforme. L'application repose largement sur un **store Redux** normalisé pour garantir la cohérence des données au sein de structures complexes. Les composants frontend sont intégrés dans un framework Angular modulaire, les workspaces Yarn servant à séparer les responsabilités. Je me suis également familiarisé avec la stratégie d'environnements utilisée pour les tests et la validation :

- un **environnement de staging** principal, alimenté par des données pseudonymisées et par le dernier code de la branche master ;
- un **environnement iso-production**, également sur données pseudonymisées, utilisé pour reproduire et déboguer les incidents de production en toute sécurité.

I.3 L'IA au service de la montée en compétences et de l'analyse

Dans la lignée de la culture d'ingénierie moderne de Padoa, j'ai intégré un IDE doté d'IA à mon travail quotidien. Mon usage de cet outil a suivi une approche pédagogique : il s'agissait d'accélérer ma compréhension du code et d'affiner mes décisions techniques, plutôt que de simplement générer du code.

I.3.1 Exploration du code et apprentissage

Naviguer dans un monorepo de grande taille aux patterns d'architecture spécifiques (comme les stores Redux normalisés) suppose une courbe d'apprentissage importante. J'ai utilisé les capacités contextuelles de l'IDE pour décortiquer les logiques complexes. Face à une bibliothèque interne ou à un composant historique que je ne connaissais pas, je demandais à l'IA d'expliquer le flux de données et la raison d'être de telle ou telle implémentation. Cela a réduit le temps passé à découvrir le code et m'a permis de me concentrer sur l'architecture d'ensemble. En demandant à l'IA de proposer des implémentations conformes aux bonnes pratiques pour des flux RxJS ou des interfaces TypeScript, j'ai pu assimiler plus vite les standards de code de l'équipe, en faisant de l'outil un véritable mentor technique disponible en permanence.

I.3.2 Soutien à la phase d'analyse

L'IA s'est révélée particulièrement utile lors du rituel d'**analyse technique et de pesée** décrit en section III. Avant de présenter un ticket à la sous-squad, je m'en servais pour identifier les cas limites : je demandais à l'IDE de confronter la solution envisagée à la logique des services existants afin de repérer d'éventuels effets de bord. Mes analyses techniques en sont devenues plus solides et la pesée des tickets plus juste. L'outil m'a aussi aidé à formuler mes choix techniques et à me préparer aux retours de l'équipe pendant les sessions d'analyse, en anticipant les problèmes potentiels de performance ou de maintenabilité.

I.3.3 Professionnalisme et bonnes pratiques

Conformément aux consignes d'intégration de Padoa, l'usage de l'IA a été encadré avec rigueur. Toutes les suggestions générées par l'IA ont été relues, testées et adaptées manuellement pour respecter la Definition of Done et les règles de lint de l'équipe. Ainsi, si l'outil a contribué à développer mes capacités, la responsabilité finale du code et de sa logique est toujours restée la mienne, en tant qu'ingénieur. Cette collaboration entre le développeur et l'IA a été un puissant catalyseur de mon passage d'une posture d'étudiant à un rôle actif d'ingénieur au sein de la squad.

II Macro-contexte & analyse de marché

Pour comprendre le rôle de Padoa, il faut examiner le périmètre socio-économique de l'écosystème français de la prévention et de la santé au travail. Ce marché est fortement réglementé et connaît une transformation numérique continue.

II.1 Le cadre français de la santé au travail (SPSTI / SPSTA)

Le système français de santé au travail se divise en deux grands modèles opérationnels :

- **SPSTI (Services de Prévention et de Santé au Travail Inter-entreprises)** : il s'agit de services mutualisés qui assurent le suivi des salariés de nombreuses petites et moyennes entreprises. En France, les SPSTI suivent plus de 16 millions de salariés, pour un budget national cumulé de 1,5 milliard d'euros [1, 4].
- **SPSTA (Services de Prévention et de Santé au Travail Autonomes)** : il s'agit de services de santé internes, dédiés exclusivement à une seule grande entreprise (généralement plusieurs milliers de salariés).

Cette répartition est le produit d'une longue histoire institutionnelle : la médecine du travail française s'est construite après la loi de 1946 autour de services financés par les employeurs, dont les acteurs, les missions et les indicateurs de suivi ont été progressivement codifiés au cours des décennies suivantes [3]. Cet héritage explique à la fois la fragmentation du secteur et l'hétérogénéité de ses systèmes d'information, que la plateforme doit prendre en compte.

Historiquement, ce secteur souffre de difficultés structurelles importantes [2] :

1. **Pénurie chronique de médecins du travail** : la baisse du nombre de médecins du travail rend impossible la réalisation de toutes les visites médicales obligatoires en présentiel dans les délais traditionnels.
2. **Absentéisme élevé et risques psychosociaux** : la hausse de l'épuisement professionnel, du stress et des problèmes de santé physique impose de passer d'une médecine curative à une évaluation proactive et préventive des risques.
3. **Dette technique importante** : la plupart des services de santé au travail reposaient traditionnellement sur des dossiers papier éparpillés ou sur des logiciels historiques installés en local, ce qui empêchait toute collaboration entre médecins, employeurs et portails salariés.

II.2 Positionnement sur le marché et stratégie d'expansion

Padoa a été fondée en 2016 pour combler ces lacunes structurelles en construisant une plateforme SaaS moderne et cloud-native. En automatisant les processus administratifs (planification, facturation, ingestion de documents) et en proposant des interfaces collaboratives aux médecins, aux entreprises et aux salariés, elle a conquis une position de leader sur le marché français, avec plusieurs millions de dossiers salariés suivis à travers les services clients.

Dans le cadre de sa stratégie de croissance, Padoa se développe selon deux axes stratégiques :

1. **Conquête des services autonomes** : passer d'une focalisation exclusive sur les services interentreprises mutualisés (SPSTI) aux grands services autonomes d'entreprise (SPSTA), qui exigent des systèmes de configuration interne dédiés. La section VIII montre la forme concrète que cela prend dans le produit : le libellé d'un écran est résolu selon le type de client, de sorte qu'un même composant nomme différemment une même entité selon que le client est un service mutualisé ou un service autonome, au lieu d'être dupliqué.
2. **Internationalisation** : conquérir des marchés hors de France, en particulier l'Allemagne. Il n'existe pas d'édition allemande distincte de la plateforme : l'essentiel du produit reste commun. La véritable distance entre les deux marchés n'est pas une liste d'options techniques, mais une manière de penser le client. Un SPSTI français est un organisme mutualisé à but non lucratif, qui raisonne en termes d'adhérents qui le rejoignent et sont représentés à son conseil d'administration ; le marché allemand aborde la même activité comme une entreprise qui vend un service à des clients. Cette différence façonne le vocabulaire et les attentes bien plus qu'elle ne fait diverger le code.

II.3 Organisation interne et place de mon équipe

La structure interne de Padoa reflète directement les deux axes stratégiques décrits ci-dessus. Elle est découpée fonctionnellement en directions couvrant les opérations clients, le produit, l'ingénierie, les ressources humaines et le revenu, la direction produit étant elle-même partagée entre le cœur de métier historique et les unités plus récentes créées pour servir les services autonomes et les marchés étrangers.

Au sein des directions technique et produit, l'ingénierie est organisée en **squads** alignées sur les pôles produit, chaque squad étant subdivisée en **sous-squads** de quatre à six personnes réunissant des développeurs, un Product Manager et un designer. Mon stage s'est déroulé dans la sous-squad responsable du cycle de vie des adhérents et des prospects, qui correspond précisément au périmètre fonctionnel des quatre études de cas présentées à partir de la section IV.

Cette organisation a eu deux conséquences pratiques sur ma façon de travailler. D'abord, une sous-squad possède son périmètre de bout en bout (migrations de base de données, services backend, packages npm partagés et front-ends Angular), si bien que chaque fonctionnalité que j'ai livrée était full-stack par construction. Ensuite, comme plusieurs sous-squads partagent le même monorepo et les mêmes packages, tout changement structurel (un nouveau type de contact, un nouveau champ d'index de recherche) doit être discuté avec les équipes voisines, et souvent challengé par elles, avant d'être mergé.

III Méthodologie agile & opérations d'ingénierie

Travailler comme ingénieur dans une entreprise technologique de grande taille exige des méthodes structurées pour maintenir un haut niveau de qualité de code et livrer de la valeur métier de façon prévisible.

III.1 Rituels agiles et cycles de sprint

La sous-squad suit un cycle de sprint structuré de deux semaines. L'équipe se coordonne à travers des rituels établis, qui assurent la transparence et le pilotage des livraisons :

- **Daily stand-up (15 min)** : chaque matin, l'équipe fait le point sur l'avancement de la veille, le programme de la journée et les éventuels points de blocage.
- **Analyse et pesée (deux fois par semaine)** : chaque membre présente une analyse technique détaillée des tickets qui lui sont attribués. L'équipe critique l'implémentation proposée, discute des cas limites et pèse collectivement le ticket sur une échelle de Fibonacci selon sa complexité technique. Chaque ticket porte à la fois le poids convenu avant le développement et le poids final enregistré ensuite, ce qui rend l'erreur d'estimation de la sous-squad mesurable plutôt qu'anecdotique.
- **Rétrospective de sprint** : à la fin de chaque cycle de deux semaines, l'équipe passe en revue ce qui a bien fonctionné et ce qui peut être amélioré, et fixe des actions concrètes pour le sprint suivant.
- **Points individuels hebdomadaires** : des points réguliers avec mon manager pour parler de développement professionnel, d'alignement et de l'avancement du stage.

Le sprint lui-même est suivi sur un tableau qui compte les tickets de l'itération en cours par statut, depuis « prêt à analyser » jusqu'à « mergé », en passant par le développement, le test et la revue. Au-delà de l'horizon du sprint, la sous-squad suit aussi ses projets pluri-sprints sur une page dédiée qui mesure l'avance ou le retard de chaque projet en jours-homme et lui attribue un responsable fixe. Ce responsable reste sur le projet pendant toute sa durée, sauf s'il est indisponible alors que le travail doit avancer, auquel cas un autre membre prend temporairement le relais.

III.2 Rotations opérationnelles

Pour préserver la stabilité de la production et l'alignement de l'équipe, deux rôles opérationnels tournent entre les ingénieurs :

III.2.1 Le rôle de support d'astreinte

Le rôle de support est attribué à un ingénieur selon une rotation hebdomadaire. Pendant cette semaine, l'ingénieur est retiré des tickets fonctionnels du sprint pour se consacrer entièrement à la bonne santé de l'application : traitement des incidents de production et campagnes de validation manuelle sur l'environnement de staging aligné sur master.

Un incident majeur résolu pendant ma rotation concernait un défaut de synchronisation asynchrone entre le statut d'un adhérent et le cycle de vie des contrats de ses salariés. Lorsqu'un adhérent était automatiquement marqué comme restreint pour impayés par le processus de relance de la facturation, son statut passait bien à « radié ». En revanche, le système ne déclenchait pas la clôture en cascade qui aurait dû suivre, et les contrats individuels des salariés restaient actifs. Pour corriger cela, j'ai refactorisé les hooks backend de modification de statut afin qu'ils déclenchent exactement le même chemin de clôture en cascade que les actions administratives manuelles. De plus, comme les données de production avaient déjà divergé, j'ai conçu un script SQL de correction, validé dans un environnement de notebook sur des structures de staging anonymisées avant d'exécuter la requête de rattrapage rétroactif en production.

Les incidents de production arrivent à la squad sous forme de tickets de support, automatiquement répliqués dans la table de tri de la sous-squad. Chaque entrée porte son origine, son statut interne et sa priorité, et une partie du rôle consiste à s'attribuer les tickets non assignés et à rediriger ceux qui relèvent d'une autre squad. Les tickets arrivent en plusieurs langues, ce qui découle de l'internationalisation décrite en section II : la plateforme sert des clients dans plusieurs pays, même si la plupart de ses fonctionnalités restent communes.

III.2.2 Le Scrum Master tournant par itération

Le rôle de Scrum Master tourne à chaque sprint (toutes les deux semaines). Le Scrum Master anime les daily stand-ups, veille à ce que le tableau de sprint reflète fidèlement l'état réel du travail, porte les dépendances de la sous-squad vis-à-vis des autres équipes et anime la rétrospective. Tenir ce rôle a déplacé mon attention de mes propres tickets vers le flux de toute la sous-squad, ce qui est l'essentiel de ce qu'il enseigne.

III.3 Cadre de découpage technique

Pour livrer les fonctionnalités de manière fiable, la sous-squad applique un cadre de découpage technique :

1. **Kick-off dev** : Product Managers, designers et ingénieurs passent en revue les maquettes ensemble, identifient les ambiguïtés d'UI et d'UX et anticipent les failles du modèle de données.
2. **Plan de découpage** : les besoins sont décomposés en une carte de dépendances de tâches techniques concrètes (migrations de base de données, hooks de services backend, montées de version de packages, composants frontend).
3. **Analyse technique** : l'ingénieur retrace le chemin des données depuis les stores d'état côté client Angular jusqu'aux tables de la base, et rédige un plan technique *avant* d'écrire la moindre ligne de code.
4. **MVP et feature flags** : les implémentations full-stack sont masquées derrière des feature flags temporaires. L'équipe peut ainsi merger tôt du code fonctionnel et l'exercer directement via l'API, avec un client HTTP, avant même que l'interface existe.

Le cycle complet de livraison est structuré autour d'une Definition of Done stricte, illustrée par le logigramme de la figure 1.

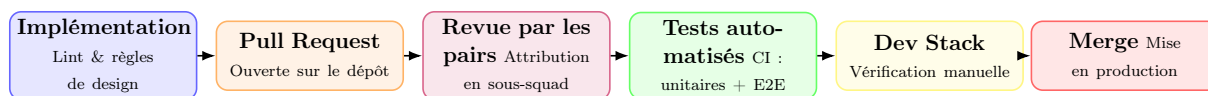


FIGURE 1 – Le cycle de la Definition of Done suivi par chaque modification livrée

IV Objectifs du stage & feuille de route

Cette section précise le cadre formel de mon stage de six mois, en distinguant la mission d'ingénierie qui m'a été confiée des cas d'implémentation technique décrits dans les sections suivantes.

IV.1 La mission : cadrage du sujet

Ma présence dans la sous-squad répondait à une mission principale : supprimer les frictions opérationnelles du cycle de vie des prospects et des adhérents. Plutôt que de développer simplement des interfaces isolées, j'avais pour tâches de :

1. faire apparaître, sur la liste des prospects, les doublons qui partagent un identifiant d'entreprise, en pré-calculant les comptes via un pipeline événementiel plutôt qu'en les calculant à l'affichage ;
2. moderniser les configurations historiques de contacts sur les portails employeur et salarié, afin que les services de santé au travail puissent exposer leur propre organisation d'équipe plutôt qu'une liste figée ;
3. vérifier et compléter les prospects créés automatiquement à partir des déclarations préalables à l'embauche, à l'aide d'un fournisseur externe d'informations légales, sans dépasser son budget de requêtes ;
4. remplacer une procédure manuelle en base de données, encore exécutée par les développeurs pour le compte du support, par une action de masse en libre-service, protégée contre les états incohérents que cette procédure pouvait produire silencieusement.

IV.2 Jalons initiaux et calendrier

Le calendrier de six mois s'articulait autour de trois jalons, afin d'aligner l'apprentissage du code et la livraison technique :

- **Jalon 1 (mois 1–2) — Intégration et synchronisation asynchrone** : mise en place de l'environnement de développement, compréhension des flux de données Redux, et implémentation des hooks d'écoute d'événements pour le suivi des doublons dans l'index de recherche.
- **Jalon 2 (mois 3–4) — Refonte du formulaire de contacts et résolution de l'assistant social** : refonte du formulaire de configuration des contacts, écriture de la requête qui résout l'assistant social titulaire d'un secteur à partir de son équipe médicale, et déploiement du type de contact correspondant sur les deux portails derrière des feature flags.
- **Jalon 3 (mois 5–6) — Enrichissement des prospects, opérations de masse et vérification collective** : branchement d'une API externe d'informations légales sur le pipeline des déclarations préalables à l'embauche, ajout du cache qui protège son budget de requêtes, transformation d'une procédure manuelle du support en action de masse en arrière-plan arbitrée par deux feature flags, et validation des pipelines lors de sessions de tests collectifs et d'audits SQL.

IV.3 Attentes fonctionnelles générales

Tous les livrables étaient soumis à des attentes fonctionnelles strictes, définies par la stratégie de croissance de Padoa :

- **Performance** : les vues de recherche et de liste doivent rester réactives quelle que soit la taille du catalogue. Elles sont servies par l'index Elasticsearch plutôt que par la base relationnelle, ce qui signifie que tout ce qu'elles affichent doit pouvoir être calculé au moment de l'indexation plutôt qu'au moment de la lecture de la page.
- **Passage à l'échelle stratégique** : le code doit supporter de grands comptes d'entreprise autonomes et rester prêt pour des fonctionnalités propres à certains marchés, en particulier l'Allemagne, sans faire diverger le socle commun.
- **Sécurité et conformité** : la plateforme traitant des données sensibles, chaque modification doit respecter strictement le Règlement Général sur la Protection des Données (RGPD), la certification française d'hébergement de données de santé (HDS) et les référentiels de l'identifiant national de santé (INS).

V Étude de cas : identification des doublons entre entités via un identifiant d'entreprise commun

Cette étude de cas détaille le développement full-stack d'un système qui identifie les fiches d'entreprise en doublon en les rapprochant sur leur SIREN. Le SIREN est l'identifiant à 9 chiffres attribué par l'INSEE à toute entreprise française ; le SIRET est l'identifiant à 14 chiffres de l'un de ses établissements, formé de ce SIREN suivi d'un numéro d'établissement à 5 chiffres. Deux fiches qui partagent un SIREN désignent donc la même entreprise juridique. Cette équivalence vaut même lorsque les deux fiches se trouvent dans des tables différentes, l'une parmi les adhérents et l'autre parmi les prospects.

Ce qui existait, et ce que j'ai ajouté : La plateforme stockait déjà les prospects et les adhérents dans deux tables distinctes, et servait déjà leurs listes à partir de l'index de recherche Elasticsearch plutôt que depuis SQL, comme pour toutes les vues de liste. Ce qu'elle ne faisait pas, c'était relier deux fiches décrivant la même entreprise juridique : un opérateur qui consultait la liste des prospects n'avait aucun moyen de savoir que l'entreprise qu'il avait sous les yeux était déjà connue ailleurs dans la plateforme. J'ai ajouté ce lien, sous la forme de deux comptes portés par chaque document prospect indexé, de la gestion d'événements qui maintient ces comptes justes lorsque les données sous-jacentes changent, et des trois éléments d'interface qui les exposent : un filtre de liste, un indicateur sur la ligne et une modale listant les doublons.

Comme pour toutes les fonctionnalités que j'ai livrées pendant le stage, le travail a été livré derrière un feature flag : un interrupteur qui garde un comportement inachevé hors de portée de tous les clients jusqu'à ce que l'équipe décide d'ouvrir la fonctionnalité à tous en production.

V.1 Phase 1 : le système existant et ses limites

Lors du bêta-test d'une fonctionnalité précédente, des utilisateurs opérationnels ont signalé qu'ils ne pouvaient pas savoir, depuis la liste des prospects, si un prospect existait déjà comme adhérent ou comme autre prospect. Faute de cette visibilité, ils perdaient du temps à qualifier des entreprises déjà présentes en base. L'équipe produit a donc demandé que l'information de doublon apparaisse directement sur la liste des prospects, accompagnée d'un filtre isolant les lignes qui ont des doublons.

V.2 Phase 2 : choix de conception

La question d'ingénierie portait sur l'endroit où les deux comptes sont calculés, et non sur le moteur qui sert la liste : comme toutes les vues de liste de la plateforme, celle-ci est déjà servie par l'index de recherche plutôt que par une requête SQL. Compter les doublons de chaque ligne au moment où une page s'affiche, que ce soit en revenant à la base relationnelle par une jointure ou en agrégeant sur tout l'index à chaque page, fait croître le coût d'affichage de cette page avec la taille du catalogue.

Nous calculons donc les deux comptes (combien de prospects et combien d'adhérents partagent l'identifiant) au moment où les données changent plutôt qu'au moment où elles sont lues, et nous les stockons dans le document Elasticsearch lui-même. Afficher une page ne coûte alors rien de plus que la lecture de deux entiers déjà présents. Le calcul réside dans une stratégie d'indexation dédiée : un handler qui reconstruit ces comptes en arrière-plan à chaque fois qu'un événement pertinent arrive. Concrètement, la requête de récupération de la stratégie assemble deux agrégats conditionnels, l'un sur les statuts de prospect et l'autre sur les statuts d'adhérent, en une seule colonne structurée, qui correspond à un objet strictement typé dans le mapping de l'index exposant les deux sous-champs entiers lus au moment de la requête.

V.3 Phase 3 : ce que j'ai implémenté

Pour garder les comptes exacts, j'ai branché cette stratégie sur la cascade d'événements. Les comptes sont recalculés de manière asynchrone lors :

- **des modifications d'établissement :** si un utilisateur modifie le SIRET d'un établissement et que la partie SIREN change réellement, l'entreprise quitte un groupe pour en rejoindre un autre. C'est seulement dans ce cas que le moteur réindexe à la fois le groupe d'origine et le groupe cible. Une modification qui laisse le SIREN inchangé ne déplace pas l'entreprise et ne déclenche pas cette cascade ;
- **des changements de statut :** lorsqu'un prospect devient adhérent actif, le bus d'événements demande à tous les documents frères partageant cet identifiant de rafraîchir leurs comptes de prospects et d'adhérents.

Côté frontend, un filtre sur la liste des prospects isole les lignes qui ont au moins un doublon, et une icône d'information apparaît sur ces lignes. Son survol affiche la répartition (combien de prospects, combien d'adhérents), le libellé suivant le compte grâce à une pluralisation internationalisée. Un clic sur le bouton d'action ouvre une modale qui liste tous les prospects et adhérents partageant l'identifiant, avec leurs informations détaillées. Le flux de données de ce pipeline de réindexation asynchrone est illustré par la figure 2.

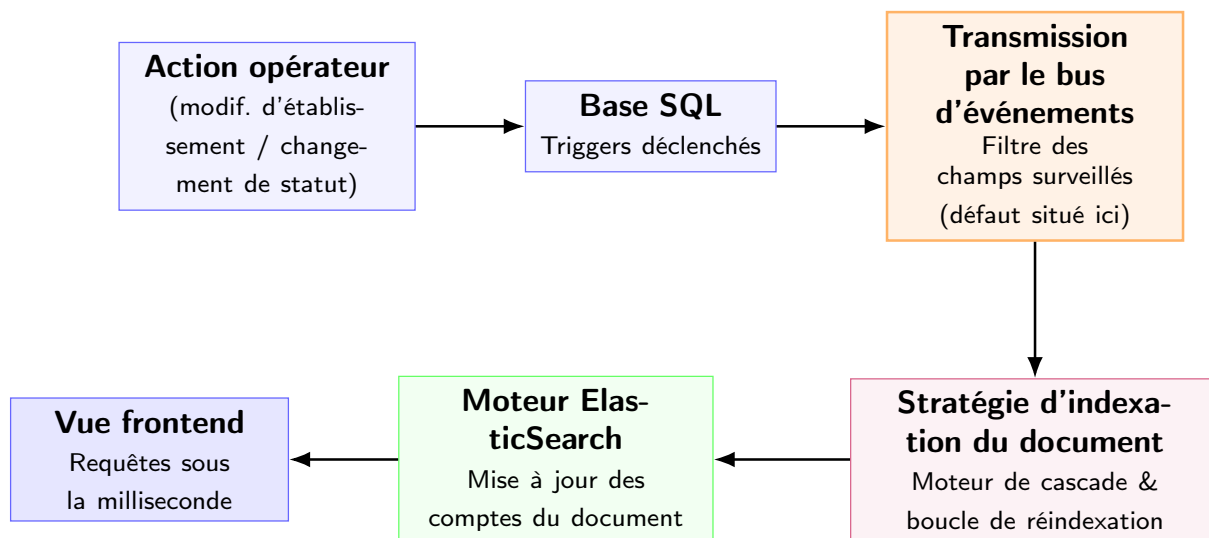


FIGURE 2 – Pipeline de réindexation asynchrone par événements. Le filtre des champs surveillés du bus d'événements est l'étape où se situait le défaut de synchronisation décrit en phase 4

V.4 Phase 4 : validation et regard critique

Les comptes affichés étant l'aboutissement d'une chaîne à trois couches (mutation SQL, bus d'événements, document Elasticsearch), valider la fonctionnalité exigeait d'inspecter les trois couches simultanément plutôt que de se fier à la seule interface. Ma boucle de vérification consistait à déclencher une mutation depuis le CRM, à suivre les actions qui en résultaient dans le journal d'actions de Redux DevTools, puis à confirmer dans le client Elasticsearch du navigateur que l'index prospect correspondant avait bien été réindexé avec les comptes attendus.

Au-delà de cette boucle individuelle, la sous-squad a mené une session de test collectif entre pairs sur différents environnements d'exécution. Cette session a mis au jour deux bugs :

- **Trou de synchronisation dans le bus d'événements** : l'historique d'un adhérent est modélisé par l'écriture d'une date de fin sur l'établissement principal, et non par la suppression ou la clôture de la ligne. Lorsque cette date était posée, les comptes de doublons des prospects frères restaient périmés, car le champ de date de fin ne faisait pas partie des champs surveillés par le bus d'événements sur la table des établissements. J'ai déclaré ce champ sur ce trigger, pour qu'il fasse partie du payload que le trigger peut transmettre et apparaisse parmi les champs surveillés correspondants, et j'ai appris à la stratégie d'indexation à lire à la fois l'ancien et le nouvel identifiant lors d'une mise à jour, de sorte que l'écriture d'une date de fin réindexe désormais les pairs.
- **Échecs de rendu de la modale** : la modale des doublons plantait à l'ouverture sur un prospect sans identifiant d'adhérent associé, parce que le client supposait que cet identifiant était toujours présent. Je l'ai rendu optionnel. Un second défaut dans le helper qui met en forme la liste ne renvoyait rien lorsque le compte du document indexé était absent ; il traite désormais un compte absent comme zéro et affiche quand même les prospects.

Les deux correctifs sont couverts par des tests de non-régression sur la stratégie d'indexation (y compris le cas où une date de fin est écrite) et sur le filtre de recherche des doublons.

Le périmètre exact de la cascade a lui-même été clarifié pendant la revue de code. Un relecteur a demandé pourquoi l'adhérent dont la date de fin vient d'être posée ne figure pas dans la liste des documents à réindexer. Il en est exclu implicitement, car le résolveur qui collecte les pairs ne conserve que les fiches encore considérées comme prospects actifs, c'est-à-dire celles dont la date de fin est encore vide. C'est le fait d'écrire ce raisonnement dans le fil de revue qui a convaincu le relecteur, et c'est une bonne illustration

d'une leçon récurrente : un invariant implicite que personne n'est capable de reformuler sur demande ne se distingue pas d'un bug.

Regard critique : Pré-calculer les comptes rend bien le coût d'affichage d'une page indépendant de la taille du catalogue, et les volumes en jeu sont de toute façon modestes : le nombre de fiches qui partagent réellement un identifiant est faible, et même un groupe d'un millier de documents resterait peu coûteux à servir pour Elasticsearch. La conception ne se justifie donc pas par le seul argument de performance, et ses vraies limites se situent ailleurs. La première est le coût du calcul d'indexation lui-même : si ce calcul est lent, la réindexation d'un seul document suffit à ajouter de la latence ou un pic de CPU sur le pipeline. La seconde est l'interface. Lorsqu'un opérateur change un statut depuis la liste, la page peut se rafraîchir avant la fin de la réindexation en arrière-plan, si bien que la ligne qu'il regarde peut être visuellement en retard sur les données qu'il vient d'écrire. C'est cette situation de concurrence, et non la taille du groupe, qui constitue le problème de cohérence avec lequel cette conception doit composer.

VI Étude de cas : résolution et publication de l'assistant social d'un secteur sur deux portails

Le domaine, en quatre phrases : Un service de santé au travail découpe son territoire en *secteurs* et rattache à chaque secteur une *équipe médicale* : un médecin du travail, des infirmiers et éventuellement des spécialistes, parmi lesquels les assistants sociaux qui accompagnent les salariés en difficulté. Chaque adhérent appartient à un secteur, donc chaque salarié hérite de l'équipe médicale du secteur de son entreprise. Le portail employeur, utilisé par les administrateurs d'une entreprise, et le portail salarié, utilisé par les salariés eux-mêmes, affichent tous deux une carte listant les contacts issus de cette équipe. Les contacts qui apparaissent sur cette carte sont configurés par le service, par statut d'entreprise, dans le panneau d'administration, en choisissant un *type de contact* pour chaque ligne.

Ce qui existait, et ce que j'ai ajouté : Il existait deux sortes de types de contact. Certains sont fixes, c'est-à-dire qu'un administrateur saisit un nom et un numéro de téléphone à la main ; d'autres sont résolus par la plateforme, qui retrouve la personne dans les données. Les médecins et les infirmiers étaient déjà résolus automatiquement. Les assistants sociaux ne l'étaient pas : un service qui voulait en publier un devait donc le saisir à la main sur chaque fiche d'entreprise, ce qui n'est pas réaliste pour un service comptant des milliers d'adhérents, et la saisie devenait obsolète dès que l'affectation changeait. J'ai ajouté un type de contact pour l'assistant social qui se résout tout seul, de l'entreprise vers son secteur, puis du secteur vers l'assistant social titulaire de son équipe, et je l'ai rendu disponible sur les deux portails. En chemin, j'ai dû lever une contrainte du formulaire d'administration qui empêchait ce type de contact de publier le moindre numéro de téléphone.

Le travail a été isolé derrière deux feature flags : l'un masquant le nouveau type de contact pendant sa construction, l'autre gouvernant la carte du portail salarié, ouverte service par service.

VI.1 Phase 1 : le système existant et ses limites

La friction comportait deux moitiés indépendantes, ce qui explique que le sujet ait été découpé en plusieurs tickets.

La première est la saisie manuelle décrite ci-dessus : la plateforme n'avait aucun moyen de désigner d'elle-même l'assistant social d'un secteur, si bien que l'information devait être recopiée sur chaque fiche d'entreprise et maintenue à la main.

La seconde est une contrainte du formulaire d'administration, et c'est la raison pour laquelle le contournement manuel n'était pas utilisable non plus. Chaque ligne de contact porte un ensemble de cases à cocher qui décident des informations réellement publiées sur les portails : le nom, l'e-mail, le numéro de téléphone. Ces cases n'étaient affichées que pour les contacts rattachés à un lieu physique, et masquées pour les contacts rattachés à un secteur. Un assistant social appartient à un secteur : aucun numéro de téléphone ne pouvait donc être publié pour lui, même saisi à la main. Publier un numéro de téléphone étant toute la raison d'être de la carte, le formulaire devait être corrigé avant que le nouveau type de contact puisse servir à quoi que ce soit.

VI.2 Phase 2 : choix de conception

Résoudre plutôt que stocker. L'affectation existe déjà dans la plateforme, au sein de l'équipe médicale d'un secteur. La recopier sur chaque fiche d'entreprise dupliquerait des milliers de fois un fait écrit une seule fois, et chaque copie pourrait ensuite devenir obsolète. Le contact est donc résolu à la lecture de la carte, en remontant de l'entreprise à son secteur, puis à son équipe.

Qu'est-ce que l'assistant social d'un secteur ? Un membre de l'équipe n'est retenu que si deux conditions sont réunies : son profil est classé comme assistant social, et son affectation à ce secteur est l'affectation titulaire et non un remplacement.

Que faire quand les données sont ambiguës ? Rien n'empêche un service de rattacher plusieurs assistants sociaux titulaires au même secteur, ce qui relève d'une erreur de configuration plutôt que d'un état voulu. La première implémentation demandait la liste complète et n'affichait un contact que si cette liste comptait exactement une entrée : un service ayant commis cette erreur ne voyait donc aucun contact, et rien n'expliquait pourquoi. Nous avons choisi le compromis inverse : la requête demande une seule ligne

et la carte affiche le premier assistant social trouvé. Un contact légèrement arbitraire est plus utile à un salarié en difficulté qu'une carte silencieusement vide.

VI.3 Phase 3 : ce que j'ai implémenté

Côté backend, la résolution est une requête dédiée écrite sous forme de Common Table Expression, c'est-à-dire une sous-requête nommée que la mécanique de contacts existante peut consommer comme n'importe quelle autre source. Elle joint l'adhérent à son secteur, puis le secteur à son équipe médicale, et conserve le membre dont l'affectation est marquée titulaire et dont le profil porte le périmètre d'assistant social. J'ai aussi assoupli la requête préexistante des contacts de secteur, qui exigeait à la fois un e-mail et un numéro de téléphone pour renvoyer quoi que ce soit, afin qu'un numéro de téléphone seul suffise.

C'est sur ce dernier changement qu'une incompatibilité de types PostgreSQL m'a bloqué un moment. Le périmètre d'un membre d'équipe est stocké sous forme de tableau de chaînes, et j'avais d'abord écrit le filtre comme une comparaison d'inclusion de tableaux (`@> ARRAY[...]`), que la base a rejetée à cause du type exact de la colonne ; un test d'appartenance à un tableau (`= ANY(...)`) correspond à la façon dont la colonne est stockée et a résolu le problème. La leçon était peu coûteuse mais réelle : du SQL brut au sein d'un code typé ne bénéficie d'aucune aide du compilateur, et seule l'exécution de la requête dit si elle est correcte.

Dans le formulaire d'administration, la contrainte décrite en phase 1 était un prédicat qui énumérait explicitement les deux types de contact liés à un lieu. Je l'ai remplacé par un helper qui répond à la question pour n'importe quel type de contact et fait passer le cas du secteur par le feature flag, transformant une énumération codée en dur en un point de décision unique.

La fonctionnalité livrée touche donc trois endroits du monorepo :

- **Panneau d'administration** : la liste déroulante des types de contact propose désormais l'assistant social de l'adhérent, avec un encart d'information précisant que la personne sera résolue automatiquement.
- **Configuration de contacts partagée** : une nouvelle catégorie dans le package de configuration des sections de contact partagé par les portails, ce qui permet à un service de faire correspondre ce type à sa propre organisation d'équipe.
- **Portail salarié** : l'assistant social résolu apparaît sur la carte de santé au travail existante, à côté du centre de rattachement et du médecin référent.

Une exigence a changé tardivement dans le sujet. Ma première implémentation reprenait la forme du contact du médecin référent, forme qui inclut une adresse e-mail professionnelle. Les revues produit et conformité ont conclu qu'un numéro de téléphone suffit et qu'une boîte mail individuelle n'a pas sa place sur le portail salarié : j'ai donc retiré l'e-mail du payload, du type et du template. La carte livrée ne porte que le nom et le numéro de téléphone de contact.

VI.4 Phase 4 : validation et regard critique

Ce qui mérite d'être testé ici, c'est la résolution plutôt que l'affichage, car la résolution dépend de données que chaque service configure lui-même, parfois d'une manière que personne n'avait anticipée. Je l'ai couverte par des tests d'intégration sur la requête (un secteur avec un assistant social titulaire, un secteur avec plusieurs, un secteur sans aucun, et un membre d'équipe assistant social sans être titulaire) ainsi que sur les règles de visibilité de la section de contacts pour chaque combinaison des deux feature flags. Une assertion supplémentaire vérifie que le payload du portail salarié ne contient plus d'adresse e-mail, afin que la décision de conformité décrite ci-dessus ne puisse pas être annulée par accident.

Regard critique : Résoudre le contact au lieu de le stocker a supprimé entièrement la saisie manuelle, et cette partie de la conception a tenu. Sa faiblesse est l'autre face du même choix : la carte n'existe que tant que l'affectation sous-jacente existe. Un secteur resté sans assistant social titulaire ne publie rien, et le salarié ne reçoit aucune explication, seulement une absence. La décision de la ligne unique de la phase 2 traite le cas ambigu dans le même esprit, en préférant une réponse imparfaite à une réponse vide, mais elle traite un symptôme. Ce qui manque encore à la fonctionnalité, c'est une solution de repli : lorsqu'aucun professionnel titulaire ne peut être résolu, la carte devrait indiquer l'accueil général du service plutôt que disparaître. Je remettrais aussi en question aujourd'hui l'absence de validation proactive côté administration, où rien n'avertit un service, au moment où il configure ce type de contact, qu'il se résout actuellement vers quelqu'un

ou non. L'outil de prévisualisation des contacts du panneau d'administration peut répondre à cette question après coup, entreprise par entreprise, mais seulement si quelqu'un pense à vérifier.

VII Étude de cas : vérification et enrichissement des prospects créés automatiquement

Ce qui existait, et ce que j’ai ajouté : Une partie de la croissance de la plateforme n’est pas d’origine commerciale. Lorsqu’une entreprise embauche un salarié, elle doit adresser une déclaration préalable à l’embauche (DPAE) à l’organisme de recouvrement des cotisations sociales, et les déclarations qui concernent chaque service de santé au travail sont récupérées selon des critères définis au niveau du service. La plateforme ingérait déjà ces fichiers et créait déjà une fiche prospect dès que l’entreprise déclarante lui était inconnue, afin que le service puisse voir qu’une entreprise de son territoire venait d’embaucher. En revanche, elle ne vérifiait pas que cette entreprise existe et soit toujours en activité, et ne complétait pas ce que la déclaration ne contient pas. Une telle déclaration ne comporte guère plus qu’un identifiant et une adresse, et en particulier aucun numéro de téléphone : la fiche arrivait donc dans la liste des prospects trop pauvre pour être exploitée. J’ai ajouté l’étape de vérification et d’enrichissement qui s’exécute désormais avant la création de cette fiche : un appel à un fournisseur externe d’informations légales, un cache local pour ne pas payer deux fois la même question, et trois issues distinctes selon la réponse du fournisseur.

L’étape est isolée derrière un unique feature flag, afin que le nouveau comportement ne puisse atteindre la production sans un déploiement délibéré.

VII.1 Phase 1 : le système existant et ses limites

Le pipeline créait un prospect à partir de toute déclaration désignant une entreprise que la plateforme ne connaissait pas encore, et prenait la déclaration pour argent comptant. Deux conséquences en découlaient. D’abord, rien ne garantissait que l’entreprise était réelle ou toujours active : un identifiant mal saisi, ou une entreprise ayant cessé son activité depuis le dépôt de la déclaration, produisait une fiche prospect comme les autres. Ensuite, cette fiche était pauvre. Une telle déclaration ne comportant aucun numéro de téléphone, les équipes du service, dont le métier est de qualifier ces entreprises et de les contacter, devaient les rechercher une par une à la main, à raison de plusieurs centaines par jour.

Le cadrage produit de cette étape a été rédigé sous forme de scénarios d’acceptation plutôt qu’en prose. Lus dans l’ordre, ils exigent qu’une déclaration désignant un établissement absent des tables d’entreprises de la plateforme déclenche une vérification de cette entreprise auprès d’une source externe ; qu’aucun prospect ne devienne visible, et qu’aucun message ne soit envoyé à l’entreprise, sur la foi d’une déclaration non vérifiée ; et qu’aucun prospect ne soit créé si la vérification ne trouve aucune entreprise pour cet identifiant, ou en trouve une qui n’est plus active. Ces trois scénarios sont précisément ce qu’implémentent les deux chemins d’abandon de la phase 3.

Le deuxième scénario mérite qu’on s’y arrête, car il décrit un état dans lequel le pipeline livré n’entre jamais. Il avait été rédigé pour une conception dans laquelle la fiche serait créée d’abord et vérifiée ensuite, ce qui laisserait une fenêtre pendant laquelle un prospect non vérifié serait visible. L’implémentation satisfait l’exigence par construction : la vérification s’exécute avant toute écriture, dans la même passe de traitement, si bien qu’il n’y a ni état intermédiaire à masquer, ni communication sortante à retenir.

VII.2 Phase 2 : choix de conception

Les informations absentes d’une déclaration (statut juridique, activité, numéro de téléphone) sont publiques, mais elles ne nous appartiennent pas : elles sont donc récupérées auprès d’un fournisseur externe d’informations légales, interrogé sur l’identifiant d’établissement porté par la déclaration.

Ce fournisseur facture à la requête, et demander un numéro de téléphone coûte plusieurs jetons : l’appeler une fois par déclaration n’était donc pas acceptable, car le même établissement apparaît dans de nombreuses déclarations, et une entreprise qui embauche dix personnes générerait dix questions identiques. Deux garde-fous précèdent donc l’appel :

1. **Ne rien demander quand le fichier répond déjà :** si la déclaration reçue contient un numéro de téléphone, aucune recherche externe n’est effectuée.
2. **Réutiliser ce que le fournisseur a déjà répondu :** avant tout appel HTTP, le moteur recherche l’établissement dans une table locale d’informations légales déjà récupérées. Une entrée encore dans sa fenêtre de validité est réutilisée telle quelle. Une entrée hors de cette fenêtre est marquée invalide, un nouvel appel est effectué, et la nouvelle réponse est réécrite avec un nouvel horodatage.

VII.3 Phase 3 : ce que j'ai implémenté

J'ai isolé ce cache derrière son propre repository de persistance, afin que l'expiration et l'invalidation soient gérées en un seul endroit. J'ai aussi remplacé un validateur de téléphone historique qui n'acceptait que le format international E.164, et rejetait donc des numéros français nationaux valides, par une expression unique qui accepte les formes nationales comme internationales. Si la valeur échoue malgré tout, le téléphone est enregistré vide et un avertissement est journalisé ; le reste des informations légales est conservé.

Lorsque le fournisseur est appelé, trois issues sont distinguées :

- **Succès** : la réponse est adaptée au format interne des informations légales. Si le fournisseur indique que l'entreprise n'est plus active, la création est abandonnée.
- **Introuvable (404 ou identifiant incohérent)** : si le fournisseur répond que l'établissement n'existe pas, ou renvoie un identifiant différent de celui demandé, la création est entièrement abandonnée, afin qu'une entreprise invérifiable ne soit jamais insérée.
- **Toute autre erreur (503, réseau, etc.)** : une défaillance temporaire est journalisée, et l'entreprise est créée à partir du contenu brut de la déclaration, sans enrichissement, afin qu'une panne du fournisseur ne bloque pas l'ingestion.

VII.4 Phase 4 : validation et regard critique

Pour valider le pipeline d'ingestion en toute sécurité, j'ai mis en place un processus de test asynchrone :

- création d'archives de test fictives contenant diverses combinaisons d'identifiants et de numéros de téléphone ;
- dépôt de ces archives dans un conteneur de stockage blob dans le cloud ;
- exécution manuelle des traitements via les interfaces en ligne de commande des conteneurs ;
- vérification de l'état de la base par l'exécution de SQL brut sur la base de staging depuis un environnement de notebook partagé, en inspectant la ligne mise en cache et ses horodatages de validité.

Chaque exécution a été lancée comme un job de workflow sur un namespace dédié aux tests entre pairs, puis auditée à travers ses logs en streaming, où apparaissent les appels sortants au fournisseur et les rejets décrits ci-dessous. L'environnement de notebook a aussi servi à forcer l'expiration d'une ligne en cache à la demande, ce qui a permis d'exercer délibérément la branche d'absence en cache plutôt que d'attendre l'expiration de la fenêtre de validité.

Les états d'exécution et les résultats observés au fil de ces exécutions sont regroupés dans la matrice de validation du tableau 1. Les identifiants d'établissement sont masqués, et les entreprises testées sont désignées par une lettre.

Source du fichier	Entreprise testée	État en entrée	Résultat attendu / assertions
Archive de smoke test	Entreprise A	Téléphone valide dans le fichier	Fiche créée immédiatement ; l'appel à l'API externe est évité.
Archive de smoke test	Entreprise B	Téléphone absent du fichier	Le fournisseur répond avec un autre établissement ; l'incohérence d'identifiant est remontée comme une erreur d'API externe de code 404 et la création est abandonnée.
Archive de smoke test	Entreprise C	Téléphone absent du fichier	L'API externe fournit le numéro ; fiche créée avec des données enrichies et vérifiées.
Archive de validation	Identifiant fictif	Téléphone absent du fichier	Le fournisseur renvoie une erreur HTTP 404 ; création entièrement abandonnée pour ne pas insérer une entreprise invérifiable.
Suite d'intégration	Entreprise C	Téléphone absent, fournisseur simulé en HTTP 503	L'échec non-404 tombe dans le traitement générique : un avertissement est journalisé et l'entreprise est créée à partir du contenu brut de la déclaration, sans enrichissement.
Archive de validation	Entreprise D	Téléphone absent du fichier	Correspondance trouvée dans le cache actif ; numéro récupéré localement depuis la table des informations légales, sans aucun frais d'API externe.

TABLE 1 – Matrice d'exécution des tests asynchrones et états de validation du pipeline. Toutes les lignes sauf une correspondent à des archives traitées sur le cluster de tests entre pairs ; la ligne de panne du fournisseur est une défaillance injectée dans la suite d'intégration, une panne transitoire ne pouvant être reproduite à la demande sur l'API réelle

Regard critique : L'architecture de cache a bien protégé le budget de jetons de l'API, et le garde-fou qui évite entièrement la recherche lorsque le fichier contient déjà un numéro de téléphone s'est révélé supprimer davantage d'appels que le cache lui-même. La gestion des erreurs a été volontairement scindée entre incohérences de données critiques et erreurs réseau : si l'API externe renvoie une erreur 404 ou un identifiant incohérent, le système abandonne proprement la création de l'entreprise afin d'éviter des fiches invalides en base. À l'inverse, si l'API subit une panne temporaire non-404, le moteur journalise un avertissement et se rabat sur la création de la fiche directement à partir du contenu de la déclaration, sans enrichissement, arbitrant ainsi entre disponibilité du système et complétude des données.

Ce que je reverrais, c'est le silence du chemin de repli. Une entreprise créée sans enrichissement parce que le fournisseur était indisponible ne se distingue plus, après coup, d'une entreprise créée avant l'existence de la fonctionnalité : rien ne la signale comme en attente d'une nouvelle tentative. Un indicateur sur la fiche, et un passage périodique sur les fiches qui le portent, combleraient ce manque pour un coût très modeste, et son absence relève d'un oubli de conception plutôt que d'un compromis délibéré.

VIII Étude de cas : révocation en masse des accès d'un utilisateur sur tous ses adhérents

Cette étude de cas décrit une fonctionnalité qui permet à un service de santé au travail de révoquer, en une seule action, tous les accès à la plateforme d'une personne qui administre plusieurs adhérents. C'est le plus important des quatre sujets, découpé en quatorze tickets de développement et livré de manière incrémentale sur plusieurs sprints.

Ce qui existait, et ce que j'ai ajouté : Un adhérent est administré sur le portail employeur par un ou plusieurs *utilisateurs employeurs*, et le modèle de données contient une ligne d'accès par personne et par entreprise. La plateforme savait déjà révoquer l'un de ces accès : un opérateur ouvrait la fiche d'une entreprise, y trouvait la personne et révoquait l'accès depuis cette fiche. Ce chemin mono-entreprise appliquait aussi les deux règles qui rendent une révocation sûre : refuser de retirer le dernier administrateur d'une entreprise, et transférer les notifications dont la personne sortante était l'unique destinataire. Ce qui n'existait pas, c'était un moyen de le faire d'un coup sur toutes les entreprises d'une même personne. J'ai construit ce chemin : deux endpoints, l'un qui prévisualise l'opération et l'autre qui la met en file d'attente, un worker en arrière-plan qui effectue les révocations par lots, et la boîte de dialogue par laquelle l'opérateur choisit le périmètre et voit ce que l'opération s'apprête à faire.

Comme dans les cas précédents, toute la fonctionnalité est isolée derrière un feature flag, défini par client, de sorte que ni l'interface ni les endpoints n'existent pour un service qui n'y a pas été délibérément inscrit.

VIII.1 Phase 1 : le système existant et ses limites

Révoquer les accès d'une personne qui administre vingt entreprises revenait à ouvrir vingt fiches d'entreprise et à répéter vingt fois la même opération. En pratique, l'opération ne restait même pas dans l'interface. L'action la plus fréquemment demandée au support client était précisément celle-ci : révoquer d'un coup tous les accès d'une personne. Le support transmettait la demande à l'équipe de développement, qui l'exécutait sous forme de requête écrite à la main depuis l'environnement de notebook partagé déjà décrit dans l'étude de cas précédente.

Ce détour avait deux coûts, et le cadrage produit les nomme tous les deux. Le premier est le temps, réparti entre l'utilisateur qui attend, l'équipe de relation client qui relaie et les développeurs qui sont interrompus. Le second, plus grave, est la justesse. Les deux règles de sécurité rappelées ci-dessus sont appliquées dans le code applicatif de la révocation mono-entreprise, et non par une contrainte de la base de données : une requête écrite à la main passe donc en dessous d'elles. Elle peut laisser une entreprise sans administrateur, ou laisser une notification dont l'unique destinataire vient d'être retiré, précisément parce qu'elle contourne le code qui l'aurait refusé.

Un détail de ce cadrage mérite d'être relevé, car il est facile de le prendre pour une contradiction : bien qu'il s'agisse de l'action de support la plus demandée, son champ d'échéance indiquait *quand on aura le temps, pas une priorité*. Chaque occurrence prise isolément était assez peu coûteuse pour être absorbée à la main ; seule l'accumulation coûtait cher, et l'accumulation n'est pas ce qu'un tableau de priorisation a tendance à voir.

VIII.2 Phase 2 : choix de conception

Quatre décisions ont façonné l'implémentation.

Exécuter le travail en arrière-plan. Une personne pouvant administrer de nombreuses entreprises, la requête ne fait que valider la demande et mettre en file une tâche asynchrone, c'est-à-dire une unité de travail prise en charge plus tard par un processus en arrière-plan, découpée en lots d'au plus cent accès sur le pool réservé aux opérations longues.

Réutiliser ou non la révocation mono-entreprise existante. C'était le véritable arbitrage, rédigé et défendu pendant le rituel d'analyse. Appeler la routine par entreprise une fois pour chaque entreprise garderait les règles métier en un seul endroit, mais répéterait pour chaque entreprise son étape la plus coûteuse, le recalcul des notifications. Une routine dédiée traitant le lot comme un seul ensemble de lignes signifie beaucoup moins d'allers-retours avec la base, mais deux chemins de code à maintenir alignés. L'équipe a choisi la seconde option en acceptant une conséquence connue : chaque comportement du chemin

par entreprise devait être rétabli sur le nouveau, chacun devenant un ticket à part planifié après le job dont il dépend.

Décider du sort du dernier administrateur restant. Révoquer le seul administrateur restant d'une entreprise la laisse sans gestionnaire. Plutôt que de coder en dur un comportement unique, la fonctionnalité lit un second feature flag préexistant, résolu pour l'utilisateur qui fait la demande et non pour le client : sans lui, ces entreprises sont exclues du lot ; avec lui, elles sont conservées et signalées à l'opérateur.

Refuser de révoquer un ensemble que l'opérateur n'a jamais vu. L'interface demande au serveur combien d'entreprises seraient concernées, affiche ce nombre et le renvoie avec la confirmation. Le serveur recalcule l'ensemble et rejette la requête si les deux ne concordent pas, ce qui protège contre un autre opérateur ou un import automatique qui modifierait les accès entre-temps.

VIII.3 Phase 3 : ce que j'ai implémenté

Côté serveur, le point d'entrée supprime les identifiants en double, demande à la base lesquels des accès demandés sont encore révocables, et ne révoque que ce sous-ensemble, chaque lot s'exécutant dans une seule transaction. La version livrée en premier exigeait au contraire que chaque identifiant soit révocable et refusait sinon le lot entier ; la phase 4 explique pourquoi il a fallu assouplir ce comportement. Avant d'écrire, le lot transfère les notifications annoncées par la boîte de dialogue : lorsque la personne révoquée est l'unique destinataire d'une notification de son entreprise, celle-ci passe aux administrateurs restants, en réutilisant telle quelle la routine qu'appelait déjà le chemin par entreprise.

La révocation ajoute à l'historique au lieu d'effacer : l'enregistrement ouvert est clos par l'écriture d'une date de fin, et un nouvel enregistrement portant le statut révoqué et l'identité de l'opérateur est inséré à côté, selon la même convention que le champ de date de fin de la première étude de cas. Rien n'est supprimé. En conséquence, la question « cet accès est-il actif aujourd'hui ? » doit être tranchée par une règle plutôt que par la lecture d'une seule colonne ; cette règle est donc écrite une seule fois sous forme de fragment SQL réutilisable : un accès est considéré comme actif soit lorsqu'un enregistrement accordé a encore une date de fin vide, soit lorsqu'aucun enregistrement d'historique n'existe, la seconde branche couvrant les lignes antérieures à la table d'historique. Un second fragment s'appuie sur le premier pour exprimer qu'une ligne porte le rôle d'administrateur alors qu'aucune autre ligne de la même entreprise ne le porte en étant active selon cette même définition. Une seule condition décide ensuite si la clause du dernier administrateur est ajoutée ou omise, ce qui permet à une même requête de servir à la fois la politique stricte et la politique permissive.

Deux endpoints s'appuient sur ce socle, l'un renvoyant les comptes affichés par la boîte de dialogue et l'autre mettant la tâche en file, tandis que la liste derrière chaque compte étend une route existante avec un paramètre optionnel plutôt que d'en ajouter une nouvelle.

La boîte de dialogue propose deux périmètres, l'entreprise actuellement ouverte par défaut ou toutes les entreprises, et déduit chaque avertissement qu'elle affiche des comptes renvoyés par le serveur plutôt que de ce que l'opérateur a saisi. Les listes derrière ces comptes sont rendues par un unique composant de présentation que j'ai extrait à cet effet, et qui est aujourd'hui réutilisé à cinq endroits. Aucun libellé ne vit dans le composant : chaque phrase est résolue selon deux axes, la langue et le type de client, de sorte que le même écran nomme l'entité administrée « adhérent » pour un service mutualisé et « unité opérationnelle » pour un service autonome. Un seul code avec une surcouverte de vocabulaire, et non une seconde version du produit.

La fin du traitement n'est pas non plus attendue de façon active. Un service partagé affichait déjà les messages de progression et de fin de toute tâche en arrière-plan qu'on lui demandait de suivre, en les poussant sur la connexion WebSocket que l'application maintient de toute façon ouverte ; cette fonctionnalité n'avait donc qu'à déclarer le type de tâche et l'utilisateur auquel le résultat est destiné. Ma première version de la boîte de dialogue suivait sa propre tâche en interrogeant le serveur sur son statut toutes les deux secondes ; l'inscription de la tâche auprès de ce service partagé a remplacé cette boucle, et le seul changement côté serveur qu'elle a exigé a été l'ajout d'une ligne désignant l'utilisateur destinataire de la notification.

VIII.4 Phase 4 : validation et regard critique

La fonctionnalité a été validée à trois niveaux plutôt que par de seuls essais manuels, car les cas intéressants sont des combinaisons de deux feature flags et de deux périmètres, fastidieuses et peu fiables à reproduire à la main. La couverture obtenue est résumée dans le tableau 2.

Niveau	Scénario testé	Résultat attendu / assertions
Worker (arrière-plan)	Lot mêlant des accès actifs et des lignes historiques sans enregistrement d'historique	Tous sont révoqués ; les lignes historiques sont traitées comme actives et ne sont pas ignorées.
Worker (arrière-plan)	Lot contenant des identifiants qui ne sont plus révocables, ou une personne supprimée entre-temps	Ces identifiants sont ignorés et les accès restants sont quand même révoqués : une entrée périmée n'annule pas le lot.
Worker (notifications)	La personne révoquée est l'unique destinataire d'une notification de son entreprise	La notification est transférée aux administrateurs restants de cette entreprise, tandis qu'une notification ayant déjà d'autres destinataires reste inchangée.
Worker (notifications)	Lot mêlant identifiants révocables et non révocables, avec des notifications rattachées aux deux	Le transfert ne s'applique qu'aux identifiants révocables, soit le même sous-ensemble que celui auquel s'applique la révocation.
API (prévisualisation)	Personne administrant plusieurs entreprises, dernier administrateur de certaines d'entre elles	Les deux comptes sont renvoyés séparément ; les entreprises dont elle est le dernier administrateur sont retirées du compte des entreprises concernées et signalées dans le second.
API (confirmation)	Le compte affiché ne correspond plus à l'ensemble réel	Requête refusée avec une erreur explicite plutôt que de révoquer un ensemble différent de celui affiché.
API (confirmation)	Personne n'ayant plus aucun accès actif	Requête refusée : une tâche vide n'est jamais mise en file.
API (deux flags)	Un demandeur disposant du flag du dernier administrateur et un demandeur sans ce flag, sur les mêmes données	Les entreprises dont la personne est le dernier administrateur sont incluses pour le premier et toujours exclues pour le second, ce qui confirme que le flag est résolu par utilisateur et non par client.
API (feature flag désactivé)	Les deux endpoints appelés sans le feature flag de protection	Les deux appels sont refusés, ce qui confirme que la fonctionnalité est entièrement inerte pour un service qui n'y est pas inscrit.
Interface	Les quatre combinaisons de périmètre et de flag du dernier administrateur	L'avertissement attendu s'affiche dans chaque cas, et l'option « toutes les entreprises » est purement désactivée lorsque le flag est inactif et qu'une seule entreprise est concernée, de sorte qu'un opérateur ne peut pas soumettre une requête que le serveur rejetterait.
Interface	Le serveur répond par l'erreur d'incohérence de compte	Le message s'affiche dans la boîte de dialogue et le compte est immédiatement redemandé, de sorte qu'une nouvelle tentative se fait sur des chiffres à jour.

TABLE 2 – Matrice de validation de la révocation en masse, couvrant le worker en arrière-plan, les deux endpoints et l'interface. Les combinaisons de flags sont couvertes par des tests automatisés plutôt que par des essais manuels, leur reproduction à la main sur un environnement partagé étant peu fiable

Regard critique : L'arbitrage de la phase 2 avait un prix, et ce qui rend cette étude de cas instructive, c'est qu'une moitié de ce prix était anticipée, tandis que l'autre était un défaut de mon fait.

La moitié anticipée est la conséquence directe de l'arbitrage. Écrire une routine de lot dédiée plutôt qu'appeler la routine par entreprise dans une boucle a rendu le lot rapide, et en contrepartie chaque

comportement du chemin par entreprise devait être rétabli explicitement sur le nouveau. L'équipe le savait au moment de choisir, et le découpage le montre : le transfert des notifications, par lequel une notification dont l'unique destinataire est révoqué passe aux administrateurs restants de l'entreprise, a été découpé dès le départ en ticket à part, planifié après le job en arrière-plan dont il dépend. L'interface a été construite en premier et ses libellés rédigés pour le comportement final, ce qui explique que la boîte de dialogue annonçait déjà le transfert avant que celui-ci n'existe. Cet ordonnancement n'est sûr que grâce au feature flag : toute la fonctionnalité reste inerte pour tout service qui n'y est pas inscrit, si bien qu'une tranche d'interface et une tranche de comportement peuvent arriver dans des sprints différents sans qu'aucun client ne voie jamais l'état intermédiaire. C'est cette façon de travailler qui permet de livrer une user story de quatorze tickets en continu plutôt qu'en une seule grosse livraison, et les libellés n'ont jamais eu à être réécrits puisqu'ils ont été rédigés une fois pour toutes, pour l'état final.

L'autre moitié n'était pas anticipée, car il s'agissait d'un défaut que j'ai introduit, et le nommer clairement est tout l'intérêt de l'évoquer. En implémentant le transfert, j'ai laissé une même règle s'écrire deux fois, dans deux langages différents. La question « lesquels de ces accès peuvent encore être révoqués ? » était tranchée sur l'endpoint par un fragment de SQL brut, et au sein du job en arrière-plan par un comptage émis via la couche de mapping objet-relationnel. Chacun paraissait correct isolément, et les deux divergeaient sur les lignes délicates, en particulier les accès dépourvus de tout enregistrement d'historique parce qu'ils sont antérieurs à cette table. Le garde-fou strict décrit en phase 3 a alors fait exactement ce pour quoi il avait été écrit, en refusant des lots entiers pour des raisons parfaitement bénignes. La correction s'est faite en deux temps : la règle vit désormais dans un module partagé que consomment à la fois les requêtes écrites à la main et celles qui sont générées, et le job ignore ce qu'il ne peut plus révoquer au lieu de tout refuser. La leçon est plus étroite que « éviter la duplication », et je préfère la garder sous cette forme étroite : une règle n'est écrite une seule fois que si tous ses appelants passent réellement par elle, et un fragment utilisé par trois requêtes sur quatre n'est pas une source unique de vérité, c'en est une imitation convaincante.

La refonte qui réunifie les chemins par entreprise et par lot, afin que la divergence n'ait plus à être gérée à la main, a depuis été écrite et relue. Reste ouverte une question de performance : le transfert demande les statistiques de notifications une fois par entreprise du lot, ce qui est tolérable pour des lots de cent, mais a été accepté lors de la pesée sans jamais être mesuré, et est devenu un ticket à part entière une fois que la revue a rendu ce coût concret. L'enregistrer plutôt que de le laisser implicite est, je crois, l'issue honnête de l'arbitrage plutôt que son échec.

IX Conclusion et bilan personnel

IX.1 Bilan professionnel & résultats collectifs

Travailler au sein d'une sous-squad responsable du cycle de vie des adhérents et des prospects m'a permis de participer à une culture d'ingénierie très collaborative. En contribuant à des modules clés comme le suivi des doublons, le contact assistant social résolu sur les deux portails, le pipeline d'ingestion des déclarations et la révocation en masse des accès aux adhérents, j'ai acquis une vision d'ensemble de l'architecture logicielle d'un produit de grande taille. Au-delà de mes études de cas individuelles, j'ai participé à des initiatives de qualité de code et de vélocité d'équipe :

- relecture quotidienne des pull requests de mes pairs, pour garantir la cohérence avec les conventions de code de l'équipe ;
- collaboration avec les Product Managers et les designers lors des réunions d'alignement de conception, pour prévenir les défauts de conception avant l'implémentation ;
- participation aux sessions de test collectif pour valider les fonctionnalités sur différents environnements utilisateurs et cas limites.

IX.2 Développement personnel & posture d'ingénieur

Mon stage de fin d'études a été une étape essentielle de mon passage du statut d'étudiant de l'École Centrale de Nantes à celui d'ingénieur logiciel autonome. Sur le plan technique, travailler dans un monorepo événementiel de grande taille a renforcé mes compétences en programmation réactive frontend, en pipelines de données asynchrones et en optimisation d'index de recherche ; cela m'a aussi confronté à une catégorie de problèmes que les projets académiques exposent rarement, à savoir la cohérence à terme entre deux couches de persistance qui ne sont jamais mises à jour dans la même transaction.

Le changement le plus significatif concerne toutefois ma posture professionnelle, et il s'est opéré en trois étapes.

- **De l'exécution des tickets à leur cadrage** : durant mes premières semaines, j'implémentais ce que décrivait le ticket. Le rituel d'analyse et de pesée m'a imposé l'habitude inverse : présenter un plan technique écrit, le défendre devant les autres membres de la sous-squad, et accepter que la solution retenue après revue ne soit souvent pas celle avec laquelle j'étais arrivé. Être challengé sur la stratégie de pré-calcul dans l'index de recherche avant d'écrire la moindre ligne de code m'a évité une réécriture, et m'a appris que l'endroit où il coûte le moins cher de se tromper, c'est le tableau blanc.
- **D'éviter les erreurs à les assumer** : le défaut de synchronisation du bus d'événements découvert lors du test collectif était, objectivement, une lacune de ma propre analyse. Le signaler explicitement à l'équipe, remonter à la cause racine plutôt que de corriger le symptôme, et écrire les tests de non-régression qui empêchent sa réapparition m'a davantage formé professionnellement que n'importe quelle fonctionnalité livrée. Je considère désormais qu'un correctif est incomplet tant que le test qui l'aurait détecté n'existe pas.
- **De la production individuelle au débit de l'équipe** : les rôles tournants de Scrum Master et de support ont déplacé mon attention de mes propres tickets vers le flux de la sous-squad : débloquer des collègues, arbitrer entre un incident de production urgent et l'engagement de sprint, et communiquer avec les Product Managers et les designers dans leur propre vocabulaire plutôt qu'en termes techniques.

Deux domaines restent pour moi des axes de progression explicites. Le premier est l'estimation : j'ai systématiquement sous-pesé les tickets lors des premiers sprints, principalement en sous-estimant le coût de la coordination entre équipes sur les packages partagés. Le second est de savoir quand arrêter de chercher seul ; j'ai compris tardivement que solliciter mon buddy après trente minutes de blocage n'est pas un manque d'autonomie, mais un bon usage du temps de l'équipe.

Dans l'ensemble, ce stage a confirmé mon souhait de travailler comme ingénieur full-stack sur des produits à la finalité sociétale tangible, et m'a donné la confiance de pouvoir porter une fonctionnalité de bout en bout, de son cadrage fonctionnel à son suivi en production, au sein d'une organisation d'ingénierie exigeante.

Références

- [1] INRS (Institut National de Recherche et de Sécurité), *Organisation des services de prévention et de santé au travail*, Publications de l'INRS, rapport technique, référence : ED-6485, Paris, mis à jour en 2023. Disponible sur <https://www.inrs.fr/demarche/services-sante-travail/organisation.html>.
- [2] Arnaud Mias, *L'horizon fuyant de la santé au travail*, SciencesPo LIEPP Policy Brief, n° 64, publications du LIEPP, Paris, mai 2024. Disponible sur <https://www.sciencespo.fr/liepp/fr/actualites/arnaud-mias-l-horizon-fuyant-de-la-sante-au-travail/>.
- [3] ADSP (Actualité et dossier en santé publique), *Santé et travail — Les origines, les acteurs et les indicateurs*, Publications de l'ADSP, Paris, ISSN : 1243-115X, n° 9, décembre 1994.
- [4] Claire Touaillat, *Quels sont les points clés du rapport 2022 sur l'activité des services de prévention et de santé au travail ?*, Éditions Lefebvre Dalloz, Paris, ISBN : 978-2-247-21650-6, juillet 2022.

X Glossaire

Ce glossaire définit les principaux termes métier, réglementaires et techniques employés dans le rapport, afin qu'il puisse être lu par une personne extérieure au projet.

Adhérent : Entreprise qui a rejoint un service de santé au travail, lequel assure en contrepartie le suivi de ses salariés.

Prospect : Entreprise qui n'a pas encore rejoint un service, soit identifiée et créée par le service lui-même (par une démarche commerciale, ou automatiquement à partir d'une déclaration préalable à l'embauche), soit créée par l'entreprise elle-même via un questionnaire d'éligibilité. Prospects et adhérents vivent dans des tables distinctes, ce qui rend nécessaire la détection des doublons de la section V.

DPAE (Déclaration Préalable à l'Embauche) :
Déclaration obligatoire qu'un employeur doit adresser à l'organisme de recouvrement des cotisations sociales avant toute embauche. Elle porte les données administratives utilisées pour créer automatiquement des fiches prospects.

SIREN (Système d'Identification du Répertoire des Entreprises) :
Numéro unique à 9 chiffres attribué par l'INSEE à toute entreprise française lors de son immatriculation. C'est l'identifiant pivot utilisé pour la détection des doublons de la section V.

SIRET (Système d'Identification du Répertoire des Établissements) :
Numéro à 14 chiffres identifiant chaque établissement physique d'une entreprise, formé du SIREN à 9 chiffres suivi d'un numéro d'établissement à 5 chiffres.

SPSTI (Service de Prévention et de Santé au Travail Interentreprises) :
Organisme à but non lucratif regroupant plusieurs entreprises pour mutualiser le suivi médical et les actions de prévention concernant leurs salariés.

SPSTA (Service de Prévention et de Santé au Travail Autonome) :
Service de santé au travail interne appartenant à une seule grande entreprise, assurant la prévention et le suivi médical de ses seuls salariés.

HDS (Hébergeur de Données de Santé) :
Certification française obligatoire pour toute entité hébergeant ou traitant des données de santé à caractère personnel, garantissant un haut niveau de sécurité et de confidentialité.

Tâche asynchrone (worker) :
Unité de travail mise en file par l'application et exécutée plus tard par un processus en arrière-plan, afin qu'une opération longue ne bloque pas la requête qui l'a déclenchée. Un lot volumineux est découpé en plusieurs jobs appartenant à la même tâche, et c'est ainsi qu'est exécutée la révocation en masse de la section VIII.

Feature flag : Technique de développement permettant d'activer ou de désactiver une fonctionnalité en production sans déployer de nouveau code. Les squads l'utilisent pour isoler le travail en cours et ouvrir une fonctionnalité client par client.

Test collectif : Rituel de test entre pairs au cours duquel toute la sous-squad exerce ensemble une fonctionnalité sur plusieurs environnements d'exécution avant sa mise en production.

Rotation de support :
Rotation hebdomadaire au cours de laquelle un ingénieur sort du travail de sprint pour traiter les incidents de production et les tickets de support.